MATHGENIUS

INTRODUCTION TO PROGRAMMING

A Practical University-Level Textbook

Foundations • Algorithms • Programming Logic • Data Structures • Software Practice

Designed for: introductory university and polytechnic courses, independent learners and beginning programmers.

First Edition • 2026

About This Textbook

This original MathGenius resource introduces core programming concepts using language-neutral explanations and illustrative Python-style examples. It supports, but does not replace, an institution's course outline, lecturer guidance, laboratory work or prescribed references.

How to study: Read each section, trace examples by hand, write and run your own version, complete the practical activities and explain your solution in your own words.

Learning Outcomes

• Explain algorithms, programs and programming languages.

• Design solutions with pseudocode, flowcharts and trace tables.

• Use variables, types, operators, input and output.

• Construct decisions, loops and reusable functions.

• Work with collections, classes, files and databases.

• Test, debug, document and plan small software projects.

• Explain introductory networking, web and security concepts.

Table of Contents

Chapter 1: Programming Foundations

What programming is
Algorithms and programs
Languages and translators
Development lifecycle
Quality and readability

Chapter 2: Algorithms, Pseudocode and Flowcharts

Problem analysis
Pseudocode
Flowcharts
Tracing and dry runs
Decomposition

Chapter 3: Variables, Types and Expressions

Variables and assignment
Primitive types
Constants and conversion
Operators and precedence
Precision and numeric data

Chapter 4: Input, Output and Validation

Input sources
Output design
Input-process-output
Files and persistence

Chapter 5: Selection and Decision Making

Boolean conditions
If and else
Multiple alternatives
Decision tables

Chapter 6: Repetition and Loops

Purpose of loops
While loops
For loops
Nested loops
Loop invariants

Chapter 7: Functions and Modular Design

Function purpose
Parameters and arguments
Return values and scope
Reuse and contracts
Testing functions

Chapter 8: Collections and Data Structures

Collections
Arrays and lists
Strings
Sets and dictionaries
Choosing structures

Chapter 9: Object-Oriented Programming

Classes and objects
Encapsulation
Inheritance and composition
Polymorphism and abstraction
Class responsibility

Chapter 10: Errors, Debugging and Testing

Error categories
Debugging method
Testing levels
Boundary and negative tests
Regression testing

Chapter 11: Files, Exceptions and Data Formats

File operations
Exceptions
CSV and JSON
Integrity and backups

Chapter 12: Database Fundamentals

Why databases
Tables and keys
SQL basics
Normalisation
Transactions

Chapter 13: Networks and Web Programming

Network basics
Client-server model
HTTP and HTTPS
HTML, CSS and JavaScript
APIs

Chapter 14: Cybersecurity and Professional Ethics

Security principles
Common vulnerabilities
Privacy
Ethics and accessibility
AI-assisted programming

Chapter 15: Software Projects and Professional Practice

Requirements
Version control
Documentation and review

Deployment and maintenance
Student project method

Chapter 1
Programming Foundations

Chapter purpose: Develop a practical understanding of programming foundations and apply it to small programming problems.

What programming is

Programming is the process of designing and expressing instructions that a computer can execute. A program is a practical implementation of a solution. Good programming begins with understanding the problem rather than immediately typing code. Identify the people who will use the system, the information it receives, the results it must produce and the constraints it must respect. Computers follow instructions precisely; they do not reliably guess what a programmer intended.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Algorithms and programs

An algorithm is a finite, ordered and unambiguous method for solving a problem. A program expresses an algorithm in a programming language. The same algorithm can be implemented in Python, Java, C or JavaScript. A correct algorithm should terminate, use steps that can be carried out, and produce the required result for the inputs within its stated scope.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Languages and translators

Machine code is directly executed by a processor. Assembly language uses symbolic instructions. High-level languages provide abstractions that make programs easier to write and maintain. A compiler translates source code into another representation, while an interpreter or runtime executes instructions. Modern language implementations may combine compilation, interpretation and runtime optimisation.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Development lifecycle

A typical lifecycle includes requirements, analysis, design, implementation, testing, deployment and maintenance. Requirements state what users need. Design plans the structure and behaviour. Implementation turns the design into source code. Testing checks the result against expectations. Maintenance corrects defects and adapts the software as needs change.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Quality and readability

Correctness means that a program meets its requirements. Readability helps people understand it. Maintainability makes future changes safer and less expensive. Use meaningful names, consistent formatting, small functions and useful comments. A program that works for one example is not necessarily correct; consider normal, boundary and invalid inputs.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in programming foundations.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 2
Algorithms, Pseudocode and Flowcharts

Chapter purpose: Develop a practical understanding of algorithms, pseudocode and flowcharts and apply it to small programming problems.

Problem analysis

Before writing code, define inputs, outputs, assumptions and constraints. For a bill calculator, inputs might include unit price and quantity; the output is the amount due. If discounts or taxes apply, define their order and boundary conditions. Ambiguous requirements should be clarified before implementation.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Pseudocode

Pseudocode describes logic without committing to a language's exact syntax. Common words include INPUT, OUTPUT, SET, IF, ELSE, WHILE and FOR. Indentation shows which instructions belong to a decision or loop. Example: INPUT price; INPUT quantity; SET total = price * quantity; OUTPUT total. The aim is clarity, not compliance with a compiler.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Flowcharts

A flowchart uses symbols to show control flow. An oval commonly represents start or end, a parallelogram input or output, a rectangle processing, and a diamond a decision. Arrows connect the steps. Label decision branches, keep the diagram readable and make sure every possible route reaches a meaningful outcome.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Tracing and dry runs

Tracing means following an algorithm manually and recording how variables change. A trace table can include the step number, inputs, variable values and output. Dry runs are especially useful for loops and nested decisions. Test more than one path, including values immediately below, at and above a decision boundary.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Decomposition

Decomposition breaks a large task into smaller parts. A student registration application might validate a student, check course prerequisites, calculate fees and save a registration. Each part can be designed and tested separately. Stepwise refinement gradually adds detail while preserving the overall goal.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in algorithms, pseudocode and flowcharts.

2. Describe a practical situation where these ideas are useful.

3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 3
Variables, Types and Expressions

Chapter purpose: Develop a practical understanding of variables, types and expressions and apply it to small programming problems.

Variables and assignment

A variable is a named reference to a value. Assignment associates a name with a value. In Python, score = 72 assigns 72 to score. Assignment is not mathematical equality: score = score + 1 means calculate a new value and store it. Names should communicate purpose, such as total_price rather than x.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Primitive types

Common types include integers, floating-point numbers, strings and Boolean values. Integers represent whole numbers; floating-point values approximate real numbers; strings represent text; Boolean values represent true or false. Types influence which operations are valid and how values are stored.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Constants and conversion

A literal is a value written directly in code. A constant is a named value intended not to change during a computation. Type conversion changes a value's representation, such as converting numeric text into an integer. Conversion can fail, so programs should handle malformed input rather than assuming it is valid.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Operators and precedence

Arithmetic operators perform calculations; comparison operators test relationships; logical operators combine conditions. Precedence determines evaluation order. For example, 2 + 3 * 4 is 14, while (2 + 3) * 4 is 20. Parentheses improve clarity when an expression could be misread.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Precision and numeric data

Computers use finite representations. Many decimal fractions cannot be represented exactly in binary floating-point, so calculations may produce tiny rounding differences. For financial calculations, decimal arithmetic or integer minor units may be preferable. Approximate values should be compared using a suitable tolerance.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in variables, types and expressions.

2. Describe a practical situation where these ideas are useful.

3. Explain one common beginner error and how to avoid it.

4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 4
Input, Output and Validation

Chapter purpose: Develop a practical understanding of input, output and validation and apply it to small programming problems.

Input sources

Programs receive data from keyboards, files, sensors, networks and other applications. Prompts should state what is expected. Input validation checks presence, type, format and range. A quantity may need to be a positive integer; a date may need a valid calendar format. Validation improves reliability but is not a complete security strategy.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Output design

Output may appear in a terminal, graphical interface, report or file. It should be accurate and understandable. Include units and labels: the number 25 is ambiguous without knowing whether it represents naira, kilograms, seconds or percent. Formatting should match the precision supported by the data.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Input-process-output

The input-process-output model identifies the information received, the transformation applied and the result produced. For Celsius-to-Fahrenheit conversion, the input is a Celsius value, the process is F = C × 9/5 + 32, and the output is Fahrenheit. Larger systems may involve repeated interaction and persistent data.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Files and persistence

Variables usually exist while a program runs. Files preserve information between runs. Programs may read text, CSV or JSON and write reports. File operations can fail because a file is missing or permissions are insufficient. Close resources safely and avoid overwriting important data without confirmation.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in input, output and validation.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 5
Selection and Decision Making

Chapter purpose: Develop a practical understanding of selection and decision making and apply it to small programming problems.

Boolean conditions

A condition evaluates to true or false. For example, score >= 50 is true when the score is at least 50. Logical AND requires both conditions to be true; OR requires at least one; NOT reverses a Boolean value. Use parentheses to make compound conditions understandable.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

If and else

An if statement executes a block when its condition is true. An else branch handles the alternative. Additional branches can test several ranges. Validate a score before assigning a grade so an impossible value is not classified as though it were valid.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Multiple alternatives

A switch or match construct selects among alternatives based on a value or pattern. It can be clearer than a long chain of repeated equality tests. Consider what happens when no case matches and provide a safe default where appropriate.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Decision tables

A decision table lists combinations of conditions and the action expected for each combination. It is useful for rules involving membership, purchase value or eligibility. It can reveal missing cases and contradictory requirements. Tests should cover every meaningful branch.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in selection and decision making.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 6
Repetition and Loops

Chapter purpose: Develop a practical understanding of repetition and loops and apply it to small programming problems.

Purpose of loops

Loops repeat instructions while a condition holds or for each item in a collection. They prevent repeated code and allow a program to process a changing amount of data. Every loop needs a clear stopping condition.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

While loops

A while loop checks its condition before each iteration. It is useful when the number of repetitions is unknown, such as asking for valid input. The loop body should update state so that termination can eventually occur. Forgetting to update a condition variable can create an infinite loop.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

For loops

A for loop iterates over a sequence or range. It is suitable for processing every item in a list or repeating a known number of times. Understand the language's range boundaries and whether the collection can safely be changed during iteration.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Nested loops

A nested loop places one loop inside another. It is useful for grids, tables and matrix operations. If the outer loop runs m times and the inner loop n times per outer iteration, the body executes m × n times. Consider the cost when data grows.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Loop invariants

A loop invariant is a property that remains true before and after each iteration. It helps explain correctness. When summing a list, the running total can represent the sum of all elements processed so far. Break and continue can be useful but should not obscure control flow.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in repetition and loops.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.

5. Suggest two test cases and state their expected outcomes.

Chapter 7
Functions and Modular Design

Chapter purpose: Develop a practical understanding of functions and modular design and apply it to small programming problems.

Function purpose

A function is a named unit of code that performs a defined task. Functions divide a program into manageable parts, reduce duplication and support testing. A function should have one clear responsibility and a predictable interface.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Parameters and arguments

Parameters are names in a function definition; arguments are values supplied when it is called. Document the expected type, range and meaning of each parameter. Validate uncertain inputs at the appropriate boundary.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Return values and scope

A return statement sends a result to the caller; printing merely displays it. Scope determines where a name can be accessed. Prefer passing required information as parameters and returning results instead of relying on many global variables.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Reuse and contracts

A function contract states what the function expects, what it produces and what conditions it guarantees. Decomposition creates components with clear responsibilities. Avoid abstractions that make a simple task harder to understand.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Testing functions

Test functions independently with normal, boundary and invalid inputs. Pure functions are particularly easy to test because the same input produces the same output without changing external state. Isolate file, network and clock dependencies when practical.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in functions and modular design.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 8
Collections and Data Structures

Chapter purpose: Develop a practical understanding of collections and data structures and apply it to small programming problems.

Collections

Collections store multiple values. Arrays, lists, tuples, sets and dictionaries are common examples. Choose according to whether order matters, duplicates are allowed and how values will be accessed.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Arrays and lists

An array stores a sequence of elements, often with indexed access. Dynamic lists can grow or shrink. Indexing commonly begins at zero, so the last valid index is one less than the collection length. Accessing an invalid index can cause an error.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Strings

Strings represent text and can often be indexed, sliced, searched, split and joined. Some languages treat strings as immutable. Unicode supports many writing systems; a visible character may consist of multiple code points, so international text requires appropriate handling.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Sets and dictionaries

A set stores unique elements and is useful for membership tests. A dictionary maps keys to values. Keys are commonly unique. A student record could map keys such as name, programme and level to values. Check whether a requested key exists.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Choosing structures

Consider the operations that dominate the task: lookup, insertion, deletion, traversal or sorting. A list is convenient for ordered items; a set helps with uniqueness; a dictionary supports key-based lookup. Memory, readability and expected data size also matter.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in collections and data structures.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.

5. Suggest two test cases and state their expected outcomes.

Chapter 9
Object-Oriented Programming

Chapter purpose: Develop a practical understanding of object-oriented programming and apply it to small programming problems.

Classes and objects

A class defines the structure and behaviour of a kind of object; an object is an instance. A Student object might store a registration number and programme and provide methods to update or display information. Classes are useful when state and behaviour belong together.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Encapsulation

Encapsulation groups data and operations and controls access to internal state. Methods can enforce rules rather than allowing arbitrary changes. Access modifiers differ by language, but the design principle is to expose a clear interface and hide unnecessary implementation details.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Inheritance and composition

Inheritance models an is-a relationship and allows a class to derive behaviour from another. Composition models a has-a relationship by combining objects. A Car contains an Engine, which is naturally represented through composition. Deep inheritance structures can be difficult to maintain.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Polymorphism and abstraction

Polymorphism allows different object types to be used through a shared interface. Several shape objects may respond to a draw operation in their own way. Abstraction exposes essential operations while hiding unnecessary details.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Class responsibility

A class should have a focused responsibility. A class that manages accounts, sends email, calculates invoices and controls backups has too many reasons to change. Split responsibilities where that improves clarity and testing.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in object-oriented programming.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.

5. Suggest two test cases and state their expected outcomes.

Chapter 10
Errors, Debugging and Testing

Chapter purpose: Develop a practical understanding of errors, debugging and testing and apply it to small programming problems.

Error categories

Syntax errors violate language rules. Runtime errors occur during execution, such as division by zero or opening a missing file. Logic errors allow execution but produce an incorrect result. Read the full error message and inspect the relevant values.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Debugging method

Reproduce the problem, record expected and actual results, reduce it to a small example, inspect values and control flow, form a hypothesis, change one thing and test again. Avoid unrelated simultaneous changes because they make diagnosis difficult.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Testing levels

Unit tests check small components; integration tests check interactions; system tests examine the complete application; acceptance tests assess whether user requirements are met. Plan tests early and record inputs, expected results and actual results.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Boundary and negative tests

Boundary tests examine values at the edges of valid ranges. For scores from zero to one hundred inclusive, test -1, 0, 1, 99, 100 and 101. Negative tests provide invalid data to check that the program fails safely.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Regression testing

Regression testing reruns previous tests after a change. It helps detect cases where a new feature or correction breaks existing behaviour. Assertions can also check assumptions during development.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in errors, debugging and testing.

2. Describe a practical situation where these ideas are useful.

3. Explain one common beginner error and how to avoid it.

4. Write a short pseudocode example demonstrating the main concept.

5. Suggest two test cases and state their expected outcomes.

Chapter 11
Files, Exceptions and Data Formats

Chapter purpose: Develop a practical understanding of files, exceptions and data formats and apply it to small programming problems.

File operations

Programs read and write files to preserve information. Text files are readable; binary files support formats such as images and compiled data. Paths may be relative to the working directory or absolute. Use safe file-handling patterns and avoid accidental overwrites.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Exceptions

An exception represents an abnormal condition. Catch selected exceptions when the program can provide a meaningful response. Silently ignoring every error hides defects. Handle errors at the level where a useful action can be taken.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

CSV and JSON

CSV represents tabular data as rows and fields. JSON represents objects, arrays, strings, numbers, Boolean values and null. Use a proper CSV parser because quoted fields may contain commas or line breaks. Validate parsed data even when the file is syntactically valid.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Integrity and backups

Data integrity means information remains accurate and consistent. Validation, constraints, transactions and backups help protect it. A backup is useful only if it can be restored, so test restoration. Never store passwords as ordinary readable text.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in files, exceptions and data formats.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 12
Database Fundamentals

Chapter purpose: Develop a practical understanding of database fundamentals and apply it to small programming problems.

Why databases

A database organises information so it can be stored, queried and updated consistently. A database management system provides structures, access control, transaction handling and recovery. Multi-user applications commonly benefit from explicit relationships and constraints.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Tables and keys

A relational database stores data in tables. A row represents a record and a column an attribute. A primary key uniquely identifies a row. A foreign key refers to a key in another table and represents a relationship.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

SQL basics

SELECT retrieves data; INSERT adds rows; UPDATE changes rows; DELETE removes rows. WHERE filters results, ORDER BY sorts and JOIN combines related tables. Use parameterised queries for user-supplied values to reduce SQL injection risk.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Normalisation

Normalisation organises data to reduce unnecessary duplication and update anomalies. First normal form requires atomic values and no repeating groups. Further normal forms address dependencies. Normalisation should support integrity and clarity, not create needless complexity.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Transactions

A transaction is a logical unit of work. ACID means atomicity, consistency, isolation and durability. A transfer between accounts should debit and credit as one transaction; a partial transfer is unacceptable. Constraints help enforce valid data.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in database fundamentals.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.

5. Suggest two test cases and state their expected outcomes.

Chapter 13
Networks and Web Programming

Chapter purpose: Develop a practical understanding of networks and web programming and apply it to small programming problems.

Network basics

A network connects devices to exchange data and share resources. IP addresses identify interfaces on IP networks. Domain names provide readable names that DNS resolves to network addresses. Communication depends on agreed protocols.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Client-server model

Clients request services and servers provide them. A browser requesting a web page is a common example. A system may separate presentation, application logic and data layers to improve maintainability.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

HTTP and HTTPS

HTTP is an application-layer protocol. Requests contain methods, targets and headers and may include a body. Responses contain status codes, headers and possibly a body. HTTPS protects HTTP communication using TLS.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

HTML, CSS and JavaScript

HTML describes page structure, CSS controls presentation and JavaScript adds browser behaviour. A static site can provide many features without a custom server. Secure shared data, secrets and cross-device accounts generally require a trusted backend or managed service.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

APIs

An API defines how software components communicate. Web APIs commonly use HTTP and JSON. Clients should handle timeouts, network failures, unexpected responses and rate limits. Private API keys must not be placed in public browser code.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in networks and web programming.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.

5. Suggest two test cases and state their expected outcomes.

Chapter 14
Cybersecurity and Professional Ethics

Chapter purpose: Develop a practical understanding of cybersecurity and professional ethics and apply it to small programming problems.

Security principles

Confidentiality limits access to authorised people; integrity protects information from improper alteration; availability keeps systems usable. Security should be considered during requirements, design, implementation, testing and operation.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Common vulnerabilities

Risks include injection, broken access control, insecure authentication, sensitive-data exposure and unsafe file handling. Validate and encode data according to context. Use parameterised queries, established authentication and least-privilege access.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Privacy

Collect personal data only for a clear purpose and protect it throughout its lifecycle. Explain what is collected, why it is needed, how long it is retained and who can access it. Avoid logging passwords, tokens or unnecessary personal details.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Ethics and accessibility

Professional programmers should respect privacy, intellectual property, safety and social impact. Accessible software considers keyboard use, readable contrast, meaningful labels and assistive technologies. Accessibility improves usability for many people.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

AI-assisted programming

AI tools can explain concepts and suggest code, but output may be incorrect or insecure. Review, test and understand generated code before using it. Do not submit credentials or confidential data unless the tool and applicable policies permit it.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in cybersecurity and professional ethics.

2. Describe a practical situation where these ideas are useful.

3. Explain one common beginner error and how to avoid it.

4. Write a short pseudocode example demonstrating the main concept.
5. Suggest two test cases and state their expected outcomes.

Chapter 15
Software Projects and Professional Practice

Chapter purpose: Develop a practical understanding of software projects and professional practice and apply it to small programming problems.

Requirements

Functional requirements describe what a system does; non-functional requirements describe qualities such as performance, usability, security and reliability. Requirements should be specific enough to test and reviewed with stakeholders.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Version control

Version control records changes to source code and supports collaboration, branching, review and recovery. A commit should represent a coherent change and have a useful message. Never commit passwords, private keys or sensitive personal information.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Documentation and review

Documentation explains installation, configuration, use and maintenance. Code review checks correctness, clarity, security and consistency. Reviews should focus on the work rather than personal criticism.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Deployment and maintenance

Deployment makes software available to users. A plan should include configuration, permissions, monitoring and recovery. Maintenance corrects defects, adapts to new environments and improves performance.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Student project method

A student project begins with a clear problem statement and manageable scope. Identify users, requirements, inputs, outputs and constraints. Design, implement a working version, test it, document results and explain limitations with evidence.

Apply it: Create a small example, trace its behaviour manually, identify one boundary case and explain how you would test it.

Chapter Review

1. Define the main terms introduced in software projects and professional practice.
2. Describe a practical situation where these ideas are useful.
3. Explain one common beginner error and how to avoid it.
4. Write a short pseudocode example demonstrating the main concept.

5. Suggest two test cases and state their expected outcomes.

Practical Laboratory Activities

For each laboratory, submit the problem statement, algorithm or pseudocode, source code, test cases, expected and actual results, and a short reflection.

Laboratory 1: Personal profile

Write a program that asks for a student's name, programme and level, then displays a formatted introduction.

1. List the inputs and their types.

2. How should an empty name be handled?

3. How could the result be saved to a file?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 2: Unit converter

Convert kilometres to miles and miles to kilometres using a named conversion constant.

1. Test zero and negative values.

2. Why use a named constant?

3. How should the result be rounded?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 3: Grade classifier

Accept a score from 0 to 100 and classify it using a stated grading scale.

1. Test 0, 49, 50, 69, 70, 100 and 101.

2. Why validate before classification?

3. How would a changed scale affect the design?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 4: Guessing game

Create a number guessing game with a limited number of attempts and useful hints.

1. Which loop is appropriate?

2. How should non-numeric input be handled?

3. How can repeated guesses be prevented?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 5: Result summary

Store scores and calculate count, total, mean, highest and lowest.

1. Which collection is suitable?

2. What happens when there are no scores?

3. How can the answer be checked manually?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 6: Reusable calculator

Create separate functions for addition, subtraction, multiplication and division.

1. What should division by zero do?

2. Why separate calculation from display?

3. How would you test each function?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 7: Contact directory

Create a small directory that stores names and phone numbers and supports search and update.

1. Would a list or dictionary be suitable?

2. How should duplicate names be handled?

3. How can data persist between runs?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 8: Library item class

Design a class for a library item with an identifier, title and availability status.

1. Which data belongs in the object?

2. Which methods change availability?

3. How do you prevent double borrowing?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 9: File report

Read records from a file, validate them and produce a summary report.

1. What if the file is missing?

2. How should malformed rows be reported?

3. Why preserve the original input?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Laboratory 10: Mini web application

Plan a browser application using HTML, CSS and JavaScript.

1. Which features can run in the browser?

2. Which data needs a trusted server?

3. What accessibility checks are needed?

Submission checklist: readable code, meaningful names, at least three test cases and one suggested improvement.

Examination-Style Practice

1. Differentiate an algorithm, a program and a programming language.
2. Explain the input-process-output model with a worked example.
3. Write pseudocode to find the largest of three numbers.
4. Explain the difference between a while loop and a for loop.
5. Describe parameter passing, return values and variable scope.
6. Compare arrays, lists, sets and dictionaries.
7. Explain encapsulation, inheritance, composition and polymorphism.
8. Describe unit, integration, system and acceptance testing.
9. Explain why parameterised SQL queries are important.
10. Discuss confidentiality, integrity and availability.
11. Explain the role of DNS and HTTPS in web communication.
12. Describe the stages of a small software development project.

Suggested Answer Guide

1. An algorithm is a finite method; a program is its implementation; a programming language provides syntax and semantics for expressing instructions.

2. Inputs are received, a process transforms them and outputs communicate the result. For example, a temperature value is converted using F = C × 9/5 + 32.

3. Read a, b and c; set largest to a; if b is greater set largest to b; if c is greater set largest to c; output largest.

4. A while loop repeats while a condition is true; a for loop iterates over a sequence or defined range.

5. Parameters name inputs, arguments supply values, return values send results to callers and scope controls where names are accessible.

6. Arrays/lists are sequences, sets hold unique values and dictionaries map keys to values.

7. Encapsulation controls access to state; inheritance derives from a base type; composition combines objects; polymorphism supports shared interfaces.

8. Unit tests check components, integration tests check interactions, system tests check the whole system and acceptance tests check user requirements.

9. Parameterised queries separate SQL structure from values and help prevent injection attacks.

10. Confidentiality controls access, integrity protects correctness and availability keeps services usable.

11. DNS resolves domain names; HTTPS protects HTTP communication using TLS.

12. A project defines requirements, designs, implements, tests, documents, deploys and maintains a solution.

Glossary of Key Terms

Algorithm: A finite, ordered and unambiguous method for solving a problem.

Argument: A value supplied to a function when it is called.

Array: An indexed collection of elements.

Boolean: A logical data type with true and false values.

Class: A definition used to create objects with related state and behaviour.

Compiler: A tool that translates source code into another representation.

Condition: An expression evaluated as true or false.

Data structure: A way of organising data to support operations.

Debugging: Locating, understanding and correcting defects.

Encapsulation: Grouping state and behaviour while controlling access.

Exception: A representation of an abnormal execution condition.

Function: A named unit of code that performs a task.

Iteration: One execution of a loop body.

Method: A function associated with an object or class.

Object: An instance with state and behaviour.

Parameter: A named input declared by a function.

Pseudocode: A language-independent description of an algorithm.

Runtime: The environment or period in which a program executes.

Scope: The region where a name can be accessed.

SQL: A language for relational database definition and queries.

Syntax: The grammatical rules of a programming language.

Variable: A named reference used to access a value.

Suggested Study Strategy

• Practise regularly by writing and running code, not only reading.

• Trace examples manually before executing them.

• Keep a record of errors, causes and corrections.

• Use your institution's syllabus to prioritise topics.

• Discuss design choices, but understand and explain your own work.

• Consult current official documentation for language versions and security guidance.

End of Textbook